

LAB-9

Course: Compiler Design Laboratory

SLOT: L23+L24

Name: Shrri Dharshan D R

Reg No: 23BPS1090

OBJECTIVE

To produce the Three Address Code (TAC) and build the Directed Acyclic Graph (DAG) for the given arithmetic expressions. This will help eliminate common sub-expressions.

PROBLEM STATEMENT

Write a C program to Generate the Three Address Code for the following expressions.

Construct the Directed Acyclic Graph (DAG) for the same expressions.

Expressions:

$$x = (a + b) * (c - d) + (e * f) - (a + b)$$

$$y = ((p + q) * r) + ((p + q) * s) + (r * s)$$

$$z = (a * b + c * d) * (e + f) + (a * b)$$

$$res = ((a + b) * (c + d)) + ((a + b) * (c + d)) + ((e + f) * (g - h))$$

AIM

To understand intermediate code generation and compiler optimization techniques by implementing:

Three Address Code (TAC)

Directed Acyclic Graph (DAG)

PROCEDURE:

1. Start the program.
2. Read the given arithmetic expressions.
3. Break the expressions into smaller sub-expressions.
- 4. For Three Address Code:**
5. Use temporary variables (t1, t2, t3, etc.).
6. Generate instructions in three-address format.
- 7. For DAG Construction:**
8. Create nodes for operands.
9. Create operator nodes.
10. Before making a new node, check if the same sub-expression already exists.
11. If it does, reuse the node to eliminate common sub-expressions.
12. Display the TAC and DAG structure.
13. Stop the program.

QUESTIONS

1. Implement the Three address code using c program for the following expressions:

- 1) $x = (a + b) * (c - d) + (e * f) - (a + b)$
- 2) $y = ((p + q) * r) + ((p + q) * s) + (r * s)$
- 3) $z = (a * b + c * d) * (e + f) + (a * b)$
- 4) $res = ((a + b) * (c + d)) + ((a + b) * (c + d)) + ((e + f) * (g - h))$

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("THREE ADDRESS CODE IMPLEMENTATION\n\n");
```

```
    /* ----- */
```

```
    /* 1) x = (a + b) * (c - d) + (e * f) - (a + b) */
```

```
    /* ----- */
```

```
    printf("Expression 1:\n");
```

```
    printf("t1 = a + b\n");    // Common subexpression (a + b)
```

```
    printf("t2 = c - d\n");    // (c - d)
```

```
    printf("t3 = t1 * t2\n");    // (a + b) * (c - d)
```

```
    printf("t4 = e * f\n");    // (e * f)
```

```
    printf("t5 = t3 + t4\n");    // (a + b)*(c - d) + (e*f)
```

```
    printf("x = t5 - t1\n\n");    // subtract (a + b)
```

```
    /* ----- */
```

```
    /* 2) y = ((p + q) * r) + ((p + q) * s) + (r * s) */
```

```
    /* ----- */
```

```
    printf("Expression 2:\n");
```

```
    printf("t1 = p + q\n");    // Common subexpression (p + q)
```

```

printf("t2 = t1 * r\n");    // (p + q) * r
printf("t3 = t1 * s\n");    // (p + q) * s
printf("t4 = r * s\n");     // (r * s)
printf("t5 = t2 + t3\n");   // ((p + q)*r) + ((p + q)*s)
printf("y = t5 + t4\n\n");  // Add (r * s)

```

```

/* ----- */
/* 3) z = (a * b + c * d) * (e + f) + (a * b) */
/* ----- */
printf("Expression 3:\n");

```

```

printf("t1 = a * b\n");    // Common subexpression (a * b)
printf("t2 = c * d\n");    // (c * d)
printf("t3 = t1 + t2\n");  // (a*b + c*d)
printf("t4 = e + f\n");    // (e + f)
printf("t5 = t3 * t4\n");  // (a*b + c*d) * (e + f)
printf("z = t5 + t1\n\n"); // Add (a * b)

```

```

/* ----- */
/* 4) res = ((a + b) * (c + d)) + ((a + b) * (c + d))
           + ((e + f) * (g - h)) */
/* ----- */
printf("Expression 4:\n");

```

```

printf("t1 = a + b\n");    // Common subexpression (a + b)
printf("t2 = c + d\n");    // (c + d)
printf("t3 = t1 * t2\n");  // (a + b) * (c + d)
printf("t4 = t3 + t3\n");  // ((a+b)*(c+d)) + ((a+b)*(c+d))
printf("t5 = e + f\n");    // (e + f)
printf("t6 = g - h\n");    // (g - h)
printf("t7 = t5 * t6\n");  // (e + f) * (g - h)
printf("res = t4 + t7\n\n"); // Final result

```

```

return 0;
}

```

OUTPUT:

```
student@student-VirtualBox:~$ nano tac.c
student@student-VirtualBox:~$ gcc tac.c -o tac
student@student-VirtualBox:~$ ./tac
THREE ADDRESS CODE IMPLEMENTATION
```

Expression 1:

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = e * f
t5 = t3 + t4
x = t5 - t1
```

Expression 2:

```
t1 = p + q
t2 = t1 * r
t3 = t1 * s
t4 = r * s
t5 = t2 + t3
y = t5 + t4
```

Expression 3:

```
t1 = a * b
t2 = c * d
t3 = t1 + t2
t4 = e + f
t5 = t3 * t4
z = t5 + t1
```

Expression 4:

```
t1 = a + b
t2 = c + d
t3 = t1 * t2
t4 = t3 + t3
t5 = e + f
t6 = g - h
t7 = t5 * t6
res = t4 + t7
```

2.Construction of DAG using c program for the following expressions:

- 1) $x = (a + b) * (c - d) + (e * f) - (a + b)$
 - 2) $y = ((p + q) * r) + ((p + q) * s) + (r * s)$
 - 3) $z = (a * b + c * d) * (e + f) + (a * b)$
- $$res = ((a + b) * (c + d)) + ((a + b) * (c + d)) + ((e + f) * (g - h))"$$

CODE:

```
#include<stdio.h>
/*
 * DAG Construction - Visual ASCII Tree Format
 * Shows / and \ branches like a real tree diagram
 *
 * 1. x  = (a + b) * (c - d) + (e * f) - (a + b)
 * 2. y  = ((p + q) * r) + ((p + q) * s) + (r * s)
 * 3. z  = (a * b + c * d) * (e + f) + (a * b)
 * 4. res = ((a+b)*(c+d)) + ((a+b)*(c+d)) + ((e+f)*(g-h))
 */

/*
=====
=====
EXPRESSION 1: x = (a+b)*(c-d) + (e*f) - (a+b)
=====
=====

DAG structure:
      (-)
     /  \
    (+) (+) <-- (a+b) SHARED
   /  \
  (*)  (*)
 /  \  /\
(+) (-) (e) (f)
 ^   ^
a  b c d
```

Note: The right child of root (-) is the SAME (+) node as the left-left grandchild. In a DAG this is one node

with two parents. We show it with [SHARED] label.

```
=====
===== */
void expr1() {
    printf("\n");

    printf("#####\n");
    printf("# Expression 1: x = (a+b)*(c-d) + (e*f) - (a+b)\n");

    printf("#####\n");

    printf("\n");
    printf("DAG Tree Structure:\n");
    printf("\n");
    printf("          (-)\n");
    printf("        /  \\\n");
    printf("      (+)  (+) [SHARED: a+b]\n");
    printf("     /  \\\n");
    printf("    (*)  (*)  +-- same node as left below\n");
    printf("   /  \\\n");
    printf("  (+) (-) (e) (f)\n");
    printf(" /  \\\n");
    printf("a  b c  d\n");
    printf("\n");
    printf(" Node List (DAG):\n");
    printf(" n0=a n1=b n2=c n3=d n4=e n5=f\n");
    printf(" n6=(+,a,b) <-- SHARED NODE (a+b)\n");
    printf(" n7=(-,c,d) (c-d)\n");
    printf(" n8=(*,e,f) (e*f)\n");
    printf(" n9=(*,n6,n7) (a+b)*(c-d)\n");
    printf(" n10=(+,n9,n8)(a+b)*(c-d) + e*f\n");
    printf(" n11=(-,n10,n6) x = n10-(a+b) [reuses n6]\n");
    printf("\n");
    printf(" Total nodes = 12 | Shared: n6=(a+b) has 2\n");
    printf("parents\n");
}
```

```
}
```

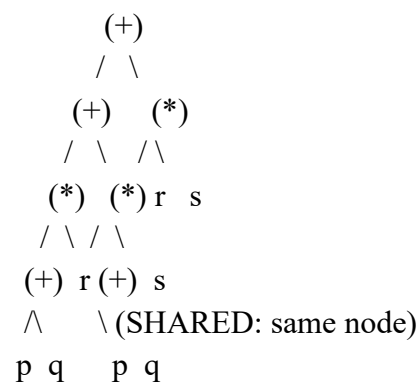
```
/*
```

```
=====
```

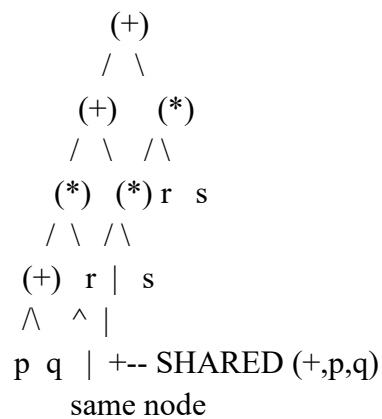
EXPRESSION 2: $y = ((p+q)*r) + ((p+q)*s) + (r*s)$

```
=====
```

DAG structure:



Simplified DAG (showing sharing):



```
===== */
```

```
void expr2() {
    printf("\n");
```

```

printf("#####\n");
#####\n");
    printf("# Expression 2: y = ((p+q)*r) + ((p+q)*s) + (r*s)
#\n");

printf("#####\n");
#####\n");
    printf("\n");
    printf("DAG Tree Structure:\n");
    printf("\n");
    printf("          (+)\n");
    printf("        /  \\\n");
    printf("      (+)  (*)\n");
    printf("    /  \  / \\\n");
    printf("  (*) (*) r  s\n");
    printf(" / \  / \ \\\n");
    printf(" (+) r (+) s\n");
    printf(" /\  ^ /\  ^\n");
    printf(" p  q | p q |\n");
    printf(" +-----+\n");
    printf("      [SHARED: same (+,p,q) node]\n");
    printf("\n");
    printf(" Node List (DAG):\n");
    printf(" n0=p n1=q n2=r n3=s\n");
    printf(" n4=(+,p,q) <-- SHARED NODE (p+q)\n");
    printf(" n5=(*,n4,r) (p+q)*r\n");
    printf(" n6=(*,n4,s) (p+q)*s [reuses n4]\n");
    printf(" n7=(*,r,s) r*s\n");
    printf(" n8=(+,n5,n6) (p+q)*r + (p+q)*s\n");
    printf(" n9=(+,n8,n7) y = n8 + r*s\n");
    printf("\n");
    printf(" Total nodes = 10 | Shared: n4=(p+q) has 2
parents\n");
}

```

```
/*
```

```
=====
```

```
=====
```

EXPRESSION 3: $z = (a*b + c*d)*(e+f) + (a*b)$

```
=====
```

DAG:

```
      (+)
     /  \
    (*)  (*) <-- (a*b) SHARED
   /  \  |
  (+) (+) +-- same (a*b) node below
  ^  ^
 (*) (*) (e) (f)
  ^  ^
 a  b c d
```

```
=====
```

```
===== */
```

```
void expr3() {
    printf("\n");
```

```
    printf("#####\n");
    printf("#####\n");
    printf("# Expression 3:  $z = (a*b + c*d)*(e+f) + (a*b)$ \n");
```

```
    printf("#####\n");
    printf("#####\n");
    printf("\n");
    printf("DAG Tree Structure:\n");
    printf("\n");
    printf("      (+)\n");
    printf("    /  \\\n");
    printf("  (*)  (*) [SHARED: a*b]\n");
    printf("    /  \\\n");
    printf("  (+) (+) +-- same node as left-left below\n");
```

```

printf("      / \ / \n");
printf("      (*) (*) (e)(f)\n");
printf("      /\ /\n");
printf("      a b c d\n");
printf("\n");
printf(" Node List (DAG):\n");
printf(" n0=a n1=b n2=c n3=d n4=e n5=f\n");
printf(" n6=(*,a,b) <-- SHARED NODE (a*b)\n");
printf(" n7=(*,c,d) (c*d)\n");
printf(" n8=(+,e,f) (e+f)\n");
printf(" n9=(+,n6,n7) (a*b)+(c*d)\n");
printf(" n10=(*,n9,n8)(a*b+c*d)*(e+f)\n");
printf(" n11=(+,n10,n6) z = n10+(a*b) [reuses n6]\n");
printf("\n");
printf(" Total nodes = 12 | Shared: n6=(a*b) has 2 parents\n");
}

```

```
/*
```

```
=====
=====
```

EXPRESSION 4: $res = ((a+b)*(c+d)) + ((a+b)*(c+d)) + ((e+f)*(g-h))$

```
=====
=====
```

DAG:

Both $(a+b)*(c+d)$ occurrences collapse to ONE node.

```

      (+)
      / \
    (+) (*)
    / \ /\
  (*) (*) (+) (-)
  | | /\
+----+ ef g h
  |
[SHARED: (a+b)*(c+d)]
/ \

```

$$\begin{array}{cc} (+) & (+) \\ \wedge & \wedge \\ a & b & c & d \end{array}$$

```

=====
===== */
void expr4() {
    printf("\n");

    printf("#####
    #####\n");
    printf("# Expression 4:
    res=((a+b)*(c+d))+((a+b)*(c+d))+((e+f)*(g-h)) #\n");

    printf("#####
    #####\n");
    printf("\n");
    printf("DAG Tree Structure:\n");
    printf("\n");
    printf("                (+)\n");
    printf("                /  \\\n");
    printf("            (+)  (*)\n");
    printf("            /  \ / \\\n");
    printf("        (*)  (*) (+) (-)\n");
    printf("        |   | \\\  /\n");
    printf("    +-----+ | e f g h\n");
    printf("        |   |\n");
    printf("    [SHARED: (a+b)*(c+d)]\n");
    printf("        /  \\\n");
    printf("        (+)  (+)\n");
    printf("        /\  /\n");
    printf("        a  b  c  d\n");
    printf("\n");
    printf(" Node List (DAG):\n");
    printf(" n0=a n1=b n2=c n3=d n4=e n5=f n6=g n7=h\n");
    printf(" n8=(+,a,b) <-- SHARED (a+b)\n");
    printf(" n9=(+,c,d) <-- SHARED (c+d)\n");
    printf(" n10=(+,e,f) (e+f)\n");

```

```

    printf(" n11=(-,g,h) (g-h)\n");
    printf(" n12=(*,n8,n9) <-- SHARED (a+b)*(c+d): BOTH
copies = this ONE node\n");
    printf(" n13=(*,n10,n11)(e+f)*(g-h)\n");
    printf(" n14=(+,n12,n12) n12+n12 [same node both
sides!]\n");
    printf(" n15=(+,n14,n13) res\n");
    printf("\n");
    printf(" Total nodes = 16 | Shared: n8,n9,n12 all have 2
parents\n");
    printf(" Without DAG a full tree would need 21 nodes.\n");
}

int main() {

printf("=====
===== \n");
    printf(" DAG CONSTRUCTION - Visual ASCII Tree (/ and
\\ style) \n");

printf("=====
===== \n");

    expr1();
    expr2();
    expr3();
    expr4();

printf("\n=====
===== \n");
    printf(" KEY: Nodes listed with indices. Same index =
SHARED node.\n");
    printf(" DAG avoids recomputing common sub-
expressions.\n");

printf("=====
===== \n");

```

```

    return 0;
}

```

OUTPUT:

```

student@student-VirtualBox:~$ nano dag_tree.c
student@student-VirtualBox:~$ gcc dag_tree.c -o dag
student@student-VirtualBox:~$ ./dag
=====
DAG CONSTRUCTION - Visual ASCII Tree (/ and \ style)
=====
#####
# Expression 1: x = (a+b)*(c-d) + (e*f) - (a+b)          #
#####
DAG Tree Structure:

      (-)
     /  \
    (+)  (+) [SHARED: a+b]
   /  \  /  \
  (*)  (*) +--- same node as left below
 /  \  /  \
(+)  (-) (e) (f)
 /  \  /  \
a   b c   d

Node List (DAG):
n0=a  n1=b  n2=c  n3=d  n4=e  n5=f
n6=(+,a,b) <-- SHARED NODE (a+b)
n7=(-,c,d)  (c-d)
n8=(*,e,f)  (e*f)
n9=(*,n6,n7) (a+b)*(c-d)
n10=(+,n9,n8) (a+b)*(c-d) + e*f
n11=(-,n10,n6) x = n10-(a+b) [reuses n6]

Total nodes = 12 | Shared: n6=(a+b) has 2 parents

#####
# Expression 2: y = ((p+q)*r) + ((p+q)*s) + (r*s)      #
#####
DAG Tree Structure:

      (+)
     /  \
    (+)  (+)
   /  \  /  \
  (*)  (*) r   s
 /  \  /  \
(+)  r (+) s
 /  \ ^  /  ^
p   q | p q |
      +-----+
      [SHARED: same (+,p,q) node]

Node List (DAG):
n0=p  n1=q  n2=r  n3=s
n4=(+,p,q) <-- SHARED NODE (p+q)
n5=(*,n4,r) (p+q)*r
n6=(*,n4,s) (p+q)*s [reuses n4]

```

```

n7=(+,r,s) r*s
n8=(+,15,n6) (p+q)*r + (p+q)*s
n9=(+,18,n7) y = n8 + r*s

Total nodes = 10 | Shared: n4=(p+q) has 2 parents

#####
# Expression 3: z = (a*b + c*d)*(e+f) + (a*b) #
#####

DAG Tree Structure:
      (+)
     /  \
    (*)  (*) [SHARED: a*b]
   /  \  /  \
  (+)  (+) (+)  (+) --- same node as left-left below
 /  \  /  \ /  \
(*)  (*) (e)(f)
 /  \ /  \
a  b c  d

Node List (DAG):
n0=a n1=b n2=c n3=d n4=e n5=f
n6=(+,1,b) <-- SHARED NODE (a*b)
n7=(+,1,d) (c*d)
n8=(+,1,f) (e+f)
n9=(+,16,n7) (a*b)+(c*d)
n10=(*,n9,n8) (a*b+c*d)*(e+f)
n11=(+,n10,n6) z = n10+(a*b) [reuses n6]

Total nodes = 12 | Shared: n6=(a*b) has 2 parents

#####
# Expression 4: res=((a+b)*(c+d))+((a+b)*(c+d))+((e+f)*(g-h)) #
#####

DAG Tree Structure:
      (+)
     /  \
    (+)  (+)
   /  \  /  \
  (*)  (*) (+)  (-)
 /  \  /  \ /  \
+-----+ | \ \ / \
|         | e f g h
|         |
[SHARED: (a+b)*(c+d)]
 /  \
(+)  (+)
 /  \ /  \
a  b c  d

Node List (DAG):
n0=a n1=b n2=c n3=d n4=e n5=f n6=g n7=h
n8=(+,1,b) <-- SHARED (a+b)
n9=(+,1,d) <-- SHARED (c+d)

```

```

      / \ |
     (+) (+) +- same node as left-left below
    / \ / \
   (+) (+) (+) (+)
  / \ / \
 a  b c  d

Node List (DAG):
n8=a n1b n2=c n3=d n4=e n5=f
n6=(+,+,b) <-- SHARED NODE (a*b)
n7=(+,+,d) (c*d)
n8=(+,+,f) (e+f)
n9=(+,n6,n7) (a*b)+(c*d)
n10=(+,n9,n8) (a*b+c*d)*(e+f)
n11=(+,n10,n6) z = n10*(a*b) [reuses n6]

Total nodes = 12 | Shared: n6=(a*b) has 2 parents

=====
# Expression 4: res=((a+b)*(c+d))*((a+b)*(c+d))*((e+f)*(g-h)) #
=====

DAG Tree Structure:

      (+)
     / \
    (+) (+)
   / \ / \
  (+) (+) (+) (+)
   | | | | |
  +---+ | e f g h
   |
 [SHARED: (a+b)*(c+d)]
   |
  (+)
 / \
a  b c  d

Node List (DAG):
n8=a n1b n2=c n3=d n4=e n5=f n6=g n7=h
n9=(+,+,b) <-- SHARED (a*b)
n9=(+,+,d) <-- SHARED (c*d)
n10=(+,e,f) (e+f)
n11=(+,g,h) (g-h)
n12=(+,n8,n9) <-- SHARED (a+b)*(c+d): BOTH copies = this ONE node
n13=(+,n10,n11) (e+f)*(g-h)
n14=(+,n12,n12) n12+n12 [same node both sides!]
n15=(+,n14,n13) res

Total nodes = 16 | Shared: n8,n9,n12 all have 2 parents
Without: DAG a full tree would need 21 nodes.

=====
KEY: Nodes listed with indices. Same index = SHARED node.
DAG avoids recomputing common sub-expressions.
=====
student@tudent-VirtualBox:~$
```

RESULT

The Three Address Code and Directed Acyclic Graph (DAG) were successfully created for the given expressions. Common sub-expressions were removed.